

GAME CLICKER REAL-TIME BERBASIS DOCKER & AWS EC2

Modul Panduan Pembuatan dan Pendeployan Aplikasi

Penyusun:

Danu Yoda Lingga Ramadhan

Agung Prayasa N.R

2026

Modul Panduan Pembuatan: Game Clicker Real-Time Berbasis Docker & AWS EC2

Modul ini berisi panduan lengkap langkah demi langkah pembuatan aplikasi game clicker interaktif secara real-time. Panduan ini mencakup arsitektur sistem, struktur proyek, kode program backend/frontend, konfigurasi Docker kontainer, hingga prosedur pendeployan di server cloud AWS EC2.

1. Arsitektur & Teknologi Stack

Aplikasi ini dirancang menggunakan arsitektur event-driven real-time dengan stack teknologi berikut: - **Backend:** **Node.js** dengan framework **Express** untuk menyajikan file statis. - **Real-Time Engine:** **Socket.io** untuk komunikasi dua arah (websocket) berlatensi rendah untuk fitur klik, obrolan (chat), dan status matchmaking. - **Database:** **JSON Flat-File Database (data/database.json)** dengan fitur auto-save setiap 5 detik agar data persisten tetap aman saat kontainer Docker direstart. - **Frontend:** **HTML5**, **Vanilla CSS3** (dengan efek neon cyberpunk, glassmorphic UI, dan partikel canvas/CSS), serta **Vanilla Javascript** untuk logika client-side. - **Kontainerisasi:** **Docker & Docker Compose** untuk portabilitas lingkungan rilis. - **Hosting / Deploy:** **AWS EC2 Instance (Ubuntu Server)** yang berjalan secara publik pada port 80.

2. Struktur Proyek

```
calm-carson/
├── data/
│   ├── database.json      # Penyimpanan data persistent (auto-generated)
│   └── public/
│       ├── client.js      # Logika frontend & handler Socket.io client
│       ├── index.html     # Tata letak antarmuka game & chat
│       └── style.css      # Styling neon cyberpunk, animasi & layout responsif
├── Dockerfile             # Konfigurasi container image Node.js alpine
├── docker-compose.yml     # Konfigurasi routing port dan volume persistent
├── package.json           # Dependensi proyek Node.js (express & socket.io)
├── server.js              # Logika server backend & Socket.io server
└── module_pembuatan.md    # Modul dokumentasi pembuatan (file ini)
```

3. Langkah-Langkah Pembuatan Proyek

Langkah 3.1: Inisialisasi Proyek Node.js Buat folder proyek baru dan inisialisasi dependensi Node.js.

```
# Membuat direktori proyek
mkdir game-clicker-realtme
cd game-clicker-realtme

# Inisialisasi package.json default
npm init -y

# Menginstal dependensi Express dan Socket.io
npm install express socket.io
```

Langkah 3.2: Konfigurasi package.json Pastikan file **package.json** mencantumkan dependensi yang diperlukan serta script start.

```
{
  "name": "tubes-cc-web-clicker",
  "version": "1.0.0",
  "description": "Real-time clicker game with matchmaking lobby",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "socket.io": "^4.6.1"
  }
}
```

4. Kode Backend Server (**server.js**)

Server bertugas mengelola koneksi socket pengguna, status permainan, obrolan, database lokal, dan hitung mundur lobby matchmaking.

Jalankan server Node Anda dengan file **server.js** berikut:

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const path = require('path');
const fs = require('fs');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

const PORT = process.env.PORT || 3000;
const DATA_DIR = path.join(__dirname, 'data');
const DATA_FILE = path.join(DATA_DIR, 'database.json');

// Memastikan folder data ada
if (!fs.existsSync(DATA_DIR)) {
  fs.mkdirSync(DATA_DIR, { recursive: true });
}

// Inisialisasi State Database
let db = {
  leaderboard: {}, // username > score
  chatHistory: [], // array of { id, username, message, timestamp, isSystem }
  lastWinner: null // { username, score, timestamp }
};
```

```

// Membaca Database lokal saat startup
if (fs.existsSync(DATA_FILE)) {
  try {
    const rawData = fs.readFileSync(DATA_FILE, 'utf8');
    db = JSON.parse(rawData);
    if (!db.leaderboard) db.leaderboard = {};
    if (!db.chatHistory) db.chatHistory = [];
    if (db.lastWinner === undefined) db.lastWinner = null;
  } catch (error) {
    console.error('Gagal memuat database, reset ke default:', error);
  }
}

// Helper penyimpanan database
let isDirty = false;
function saveDatabase() {
  if (!isDirty) return;
  try {
    fs.writeFileSync(DATA_FILE, JSON.stringify(db, null, 2), 'utf8');
    isDirty = false;
    console.log('Database berhasil disimpan otomatis.');
```

```

}

// Auto-save setiap 5 detik jika ada perubahan data
const saveInterval = setInterval(saveDatabase, 5000);

// Penanganan Graceful Shutdown
function shutdown() {
  console.log('Server dimatikan. Menyimpan database...');
  clearInterval(saveInterval);
  saveDatabase();
  process.exit(0);
}
process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);

// Sajikan file statis dari folder public
app.use(express.static(path.join(__dirname, 'public')));

// Variabel Kontrol Real-Time Online & Turnamen
const onlineUsers = new Map(); // socket.id & username
let isLobbyActive = false;
let lobbyTimeLeft = 0;
let tournamentParticipants = []; // Maks 3
let lobbyInterval = null;
```

```

let isTournamentActive = false;
let tournamentTimeLeft = 0;
let tournamentScores = {}; // username -> click count
let tournamentInterval = null;

function getLeaderboard() {
  return Object.entries(db.leaderboard)
    .map(([username, score]) => ({ username, score }))
    .sort((a, b) => b.score - a.score)
    .slice(0, 10);
}

// Mengaktifkan Event Socket.io
io.on('connection', (socket) => {
  console.log('Pengguna terhubung:', socket.id);

  // Kirim kondisi awal server ke client baru
  socket.emit('init', {
    leaderboard: getLeaderboard(),
    chatHistory: db.chatHistory.slice(-50),
    onlineCount: io.engine.clientsCount,
    lastWinner: db.lastWinner,
    lobbyState: { isActive: isLobbyActive, timeLeft: lobbyTimeLeft, participants: tournamentParticipants },
    tournamentState: {
      isActive: isTournamentActive,
      timeLeft: tournamentTimeLeft,
      participants: tournamentParticipants,
      scores: tournamentScores
    }
  });

  io.emit('onlineCount', io.engine.clientsCount);

  // Pendaftaran Username
  socket.on('join', (username) => {
    const cleanUsername = username.trim().substring(0, 20) || 'Anonymous';
    onlineUsers.set(socket.id, cleanUsername);

    if (db.leaderboard[cleanUsername] === undefined) {
      db.leaderboard[cleanUsername] = 0;
      isDirty = true;
    }

    io.emit('leaderboardUpdate', getLeaderboard());

    // Broadcast pesan sistem ke chat
    const systemMsg = {
      id: 'sys-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),
      username: 'System',
      message: `${cleanUsername} bergabung ke server!`,

```

```

        timestamp: Date.now(),
        isSystem: true
    };
    db.chatHistory.push(systemMsg);
    if (db.chatHistory.length > 100) db.chatHistory.shift();
    isDirty = true;
    io.emit('chatMessage', systemMsg);
});

// Event Klik Core
socket.on('click', () => {
    const username = onlineUsers.get(socket.id);
    if (!username) return;

    db.leaderboard[username] = (db.leaderboard[username] || 0) + 1;
    isDirty = true;

    // Tambahkan skor turnamen jika peserta terdaftar
    if (isTournamentActive && tournamentParticipants.includes(username)) {
        tournamentScores[username] = (tournamentScores[username] || 0) + 1;
        io.emit('tournamentScoresUpdate', tournamentScores);
    }

    io.emit('leaderboardUpdate', getLeaderboard());
});

```

```

// Obrolan Chat
socket.on('chatMessage', (messageText) => {
    const username = onlineUsers.get(socket.id);
    if (!username) return;

    const cleanMsg = messageText.trim().substring(0, 200);
    if (cleanMsg.length === 0) return;

    const chatMsg = {
        id: 'msg-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),
        username,
        message: cleanMsg,
        timestamp: Date.now(),
        isSystem: false
    };

    db.chatHistory.push(chatMsg);
    if (db.chatHistory.length > 100) db.chatHistory.shift();
    isDirty = true;
    io.emit('chatMessage', chatMsg);
});

// Fitur Unsend Pesan milik sendiri
socket.on('unsendMessage', (messageId) => {

```

```

const username = onlineUsers.get(socket.id);
if (!username) return;

const index = db.chatHistory.findIndex(msg => msg.id === messageId);
if (index === -1) return;

const msg = db.chatHistory[index];
if (msg.username === username) {
  db.chatHistory.splice(index, 1);
  isDirty = true;
  io.emit('messageDeleted', messageId);
}
});

// Admin: Hapus Chat
socket.on('adminDeleteMessage', ({ messageId, password }) => {
  if (password !== (process.env.ADMIN_PASSWORD || 'admin123')) {
    socket.emit('adminError', 'Password admin salah!');
    return;
  }
  const index = db.chatHistory.findIndex(msg => msg.id === messageId);
  if (index === -1) return;

  db.chatHistory.splice(index, 1);
  isDirty = true;

```

```

  io.emit('messageDeleted', messageId);
});

// Admin: Reset Skor Pemain
socket.on('adminDeletePlayer', ({ targetUsername, password }) => {
  if (password !== (process.env.ADMIN_PASSWORD || 'admin123')) {
    socket.emit('adminError', 'Password admin salah!');
    return;
  }
  if (db.leaderboard[targetUsername] !== undefined) {
    delete db.leaderboard[targetUsername];
    isDirty = true;
    io.emit('leaderboardUpdate', getLeaderboard());
  }

  const systemMsg = {
    id: 'sys-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),
    username: 'System',
    message: `Pemain "${targetUsername}" telah dihapus oleh Admin.`,
    timestamp: Date.now(),
    isSystem: true
  };
  db.chatHistory.push(systemMsg);
  io.emit('chatMessage', systemMsg);
}
});

```

```
// Membuka Lobby Turnamen (Umum/Public)
socket.on('openTournamentLobby', () => {
  if (isTournamentActive || isLobbyActive) {
    socket.emit('tournamentError', 'Turnamen atau Lobby sedang berjalan!');
    return;
  }

  isLobbyActive = true;
  lobbyTimeLeft = 15;
  tournamentParticipants = [];
  isDirty = true;

  io.emit('lobbyOpened', { timeLeft: lobbyTimeLeft, participants: tournamentParticipants });

  const systemMsg = {
    id: 'sys-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),
    username: 'System',
    message: '■ Lobby turnamen klik dibuka! Silakan gabung (maksimal 3 pemain). ■',
    timestamp: Date.now(),
    isSystem: true
  };
  db.chatHistory.push(systemMsg);
  io.emit('chatMessage', systemMsg);
```

```
// Timer Hitung Mundur Lobby 15 Detik
lobbyInterval = setInterval(() => {
  lobbyTimeLeft--;
  io.emit('lobbyTick', lobbyTimeLeft);

  if (lobbyTimeLeft <= 0) {
    clearInterval(lobbyInterval);
    isLobbyActive = false;

    // Validasi minimal 2 pemain
    if (tournamentParticipants.length >= 2) {
      startTournament();
    } else {
      io.emit('lobbyCanceled', 'Lobby dibatalkan karena kekurangan pemain (minimal 2 pemain).');
      tournamentParticipants = [];

      const sysMsg = {
        id: 'sys-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),
        username: 'System',
        message: '■■ Turnamen dibatalkan karena jumlah peserta kurang dari 2.',
        timestamp: Date.now(),
        isSystem: true
      };
      db.chatHistory.push(sysMsg);
      io.emit('chatMessage', sysMsg);
```



```

    }
  }, 1000);
});

// Gabung ke Lobby Turnamen
socket.on('joinTournamentLobby', () => {
  const username = onlineUsers.get(socket.id);
  if (!username || !isLobbyActive) return;

  if (tournamentParticipants.includes(username)) return;

  if (tournamentParticipants.length >= 3) {
    socket.emit('tournamentError', 'Lobby sudah penuh!');
    return;
  }

  tournamentParticipants.push(username);
  io.emit('lobbyUpdate', tournamentParticipants);

  // Mulai instan jika slot penuh (3 pemain)
  if (tournamentParticipants.length === 3) {
    clearInterval(lobbyInterval);
    isLobbyActive = false;
    startTournament();
  }
});

```

```

  }
});

// Putus Koneksi
socket.on('disconnect', () => {
  const username = onlineUsers.get(socket.id);
  if (username) {
    onlineUsers.delete(socket.id);
    const systemMsg = {
      id: 'sys-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),
      username: 'System',
      message: `${username} meninggalkan server.`,
      timestamp: Date.now(),
      isSystem: true
    };
    db.chatHistory.push(systemMsg);
    io.emit('chatMessage', systemMsg);
  }
  io.emit('onlineCount', io.engine.clientsCount);
});

// Memulai Game Turnamen
function startTournament() {
  isTournamentActive = true;
}

```

```

tournamentTimeLeft = 60;
tournamentScores = {};

tournamentParticipants.forEach(p => {
  tournamentScores[p] = 0;
});
isDirty = true;

io.emit('tournamentStart', { timeLeft: tournamentTimeLeft, participants: tournamentParticipants });

const systemMsg = {
  id: 'sys-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),
  username: 'System',
  message: `■ Turnamen Klik Dimulai! Peserta: ${tournamentParticipants.join(', ')}. Klik secepat mungkin! ■`,
  timestamp: Date.now(),
  isSystem: true
};
db.chatHistory.push(systemMsg);
io.emit('chatMessage', systemMsg);

// Hitung mundur turnamen 1 menit (60 detik)
tournamentInterval = setInterval(() => {
  tournamentTimeLeft--;
  io.emit('tournamentTick', tournamentTimeLeft);

```

```

if (tournamentTimeLeft <= 0) {
  clearInterval(tournamentInterval);
  isTournamentActive = false;

  // Cari Pemenang
  let winnerName = null;
  let highestScore = 0;
  Object.entries(tournamentScores).forEach(([username, score]) => {
    if (score > highestScore) {
      highestScore = score;
      winnerName = username;
    }
  });

  if (winnerName) {
    db.lastWinner = {
      username: winnerName,
      score: highestScore,
      timestamp: Date.now()
    };
    isDirty = true;
    io.emit('tournamentEnd', db.lastWinner);

    const winMsg = {
      id: 'sys-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9),

```

```

      username: 'System',
      message: `■ Turnamen Selesai! Pemenangnya adalah "${winnerName}" dengan ${highestScore} klik!`,
      timestamp: Date.now(),
      isSystem: true
    };
    db.chatHistory.push(winMsg);
    io.emit('chatMessage', winMsg);
  } else {
    io.emit('tournamentEnd', null);
  }
  tournamentParticipants = [];
}
}, 1000);
}

server.listen(PORT, () => {
  console.log(`Server berjalan di port ${PORT}`);
});

```